I-DLV-sr

A Stream Reasoning System based on I-DLV

Francesco Calimeri, Marco Manna, Elena Mastria, Maria Concetta Morelli, Simona Perri, Jessica Zangari

I-DLV-sr components

- I^2-DLV
- Apache Flink

Syntax and Semantics

Syntax

- ASP-Core-2
- normal, stratified programs
- streaming literals

Streaming literals

- a at least c in $\{d_1, ..., d_m\}$
- a always in c in $\{d_1, ..., d_m\}$
- a count c in $\{d_1, ..., d_m\}$

Some shortcuts:

- a at least 1 in $\{d_1, ..., d_m\}$ => a in $\{d_1, ..., d_m\}$
- a at least 1 in 0 => a
- "not" a at least in c in $\{d_1, ..., d_m\}$ => a at most c' in $\{d_1, ..., d_m\}$
  - where $c = c' + 1$
- $[n]$ instead of $\{0, \ldots, n\}$

Rules

- $a :- l_1, \ldots, l_n$
- $\#temp\ a :- l_1, \ldots, l_n$

- $irregular :-$ not $tram(X, "St1")$ in $[10]$.
- $\#$temp $num\_anomalies(X) :-$ irregular count X in $[30]$.
- $alert :- num\_anomalies(X), X > 5$.

Semantics of streaming literals

- Stream $\Sigma = \langle S_0, \ldots, S_n \rangle$
  - S_i: set of ground predicate atoms
  - $a \in S_i$: a is true at time point i
  - $\Sigma = \{\{tram(\text{"1"}, \text{"St1"})\}, \{\}, \{tram(\text{"WLB"}, \text{"St1"}), tram(\text{"62"}, \text{"St1"})\}\}$
- Backward Observation $O(\Sigma, D)$ where $D = \{d_1, \ldots, d_m\} \subset \mathbb{N}$
  - $O(\Sigma, D) = \{S_i | i = n - d \wedge d \in D \wedge i \geq 0\}$

- $O(\Sigma, \{0\}) = \{\{tram(\text{"WLB"}, \text{"St1"}), tram(\text{"62"}, \text{"St1"})\}\}$
- $O(\Sigma, \{1\}) = \{\{\}\}$
- $O(\Sigma, \{0, 2\}) = \{\{tram(\text{"1"}, \text{"St1"})\}, \{tram(\text{"WLB"}, \text{"St1"}), tram(\text{"62"}, \text{"St1"})\}\}$

Semantics of streaming literals

- a at least c in $\{d_1, ..., d_m\}$
  - $|\{A \in O(\Sigma, D) : a \in A\}| \geq c$
- a always in c in $\{d_1, ..., d_m\}$
  - $\forall A \in O(\Sigma, D), a \in A$
- a count c in $\{d_1, ..., d_m\}$
  - $|\{A \in O(\Sigma, D) : a \in A\}| = c$

- $\{\{tram(”1”, ”St1”)\}, \{tram(”1”, ”St1”)\}, \{tram(”WLB”, ”St1”), tram(”62”, ”St1”)\}\}$
- tram("1", Y) atleast 2 in [2]
- tram("1", "St1") always in $\{1, 2\}$
- tram("WLB", Y) count 1 in [2]

Semantics of streaming literals

- *Substitution* σ
  - mapping from variables to constants
  - for atom a: $\sigma(a)$
  - for literal l: $\sigma(l)$
- *applicable* substitution
  - r, Σ: r is applicable on $\Sigma \iff \exists \sigma : \forall b \in B(r) : \Sigma \models \sigma(b)$

- $\Sigma = \{\{tram(1, St1)\}\}$
- $r : arrivalAt(Y) :- tram(X, Y)$
- $\sigma = \{X \to 1, Y \to St1\}$

Semantics of streaming literals

- *trigger* $\langle \sigma, r \rangle$ on Σ
  - r is applicable on Σ via σ
- an *application* of $\langle \sigma, r \rangle$ on Σ
  - $\Sigma = \{S_0, \ldots, S_n\}, a = H(r)$
  - $\Sigma \langle \sigma, r \rangle \Sigma'$
    - $\Sigma' = \{S_0, \ldots, S_n \cup \sigma(a)\}$

- $\Sigma = \{\{tram(1, St1)\}\}$
- $r : arrivalAt(Y) :- tram(X, Y)$
- $\sigma = \{X \to 1, Y \to St1\}$
- $\langle \sigma, r \rangle \sigma = \{\{tram(1, St1), arrival(St1)\}\}$

Stratification

- P is stratified by a disjoint set of rules $\Pi_1, \ldots, Pi_k$
  - $P = \Pi_1 \cup \ldots \cup \Pi_k$

1. for each harmless literal in the body of a rule in Π_i with predicate p
   - $\{r \in P | H(r) = p(t_1, \ldots, t_n)\} \subseteq \cup_{j=1}^{i} \Pi_j$
2. for each non-harmless literal in the body of a rule in Π_i with predicate p
   - $\{r \in P | H(r) = p(t_1, \ldots, t_n)\} \subseteq \cup_{j=1}^{i-1} \Pi_j$

Stratum application

- P *stratified* by $\Pi_1, \ldots, \Pi_k$, Stream Σ
- *Stratum application* of Π_s (for $s \in \{1, \ldots, k\}$) is
  - finite sequence of streams $\Sigma_0, \ldots, \Sigma_h$ where $\Sigma_0 = \Sigma$
  - for $h \geq 0$:

- for each $0 \leq i < h$, there is a trigger $\langle r_i, \sigma_i \rangle$ for Π_s such that $\Sigma_i \langle r_i, \sigma_i \rangle \Sigma_{i+1}$
- for each $0 \leq i < j < h$, if $\Sigma_i \langle r_i, \sigma_i \rangle \Sigma_{i+1}$, $\Sigma_j \langle r_j, \sigma_j \rangle \Sigma_{j+1}$ and $r_i = r_j$ then $\sigma_i \neq \sigma_j$
- there is no trigger $\langle \sigma, r \rangle$ for Π_s on Σ_h such that $\langle \sigma, r \rangle \notin \{\langle r_i, \sigma_i \rangle\}_{0 \leq i \leq h}$

- Σ_h: *outcome*

Restricted Streaming model

- P *stratified* by $\Pi_1, \ldots, \Pi_k$
- Stream $\Sigma = \langle S_0, \ldots, S_n \rangle$

- $\Sigma_{\Pi_1} = outcome(\Pi_1, \Sigma)$
- $\Sigma_{\Pi_i} = outcome(\Pi_i, \Sigma_{\Pi_{i-1}})$
- $\mathcal{R}(P, \Sigma) = \Sigma_{\Pi_k}$: Outcome over strata

- $\Sigma' = \langle S'_0, \ldots, S'_{n-1}, S_n \rangle$
- $S'_0 = \mathcal{R}(P, \langle S_0 \rangle)$
- $S'_i = \mathcal{R}(P, \langle S'_0, \ldots, S'_{i-1}, S_i \rangle)$
- restricted streaming model $\mathcal{R}(P, \Sigma')$

Streaming model

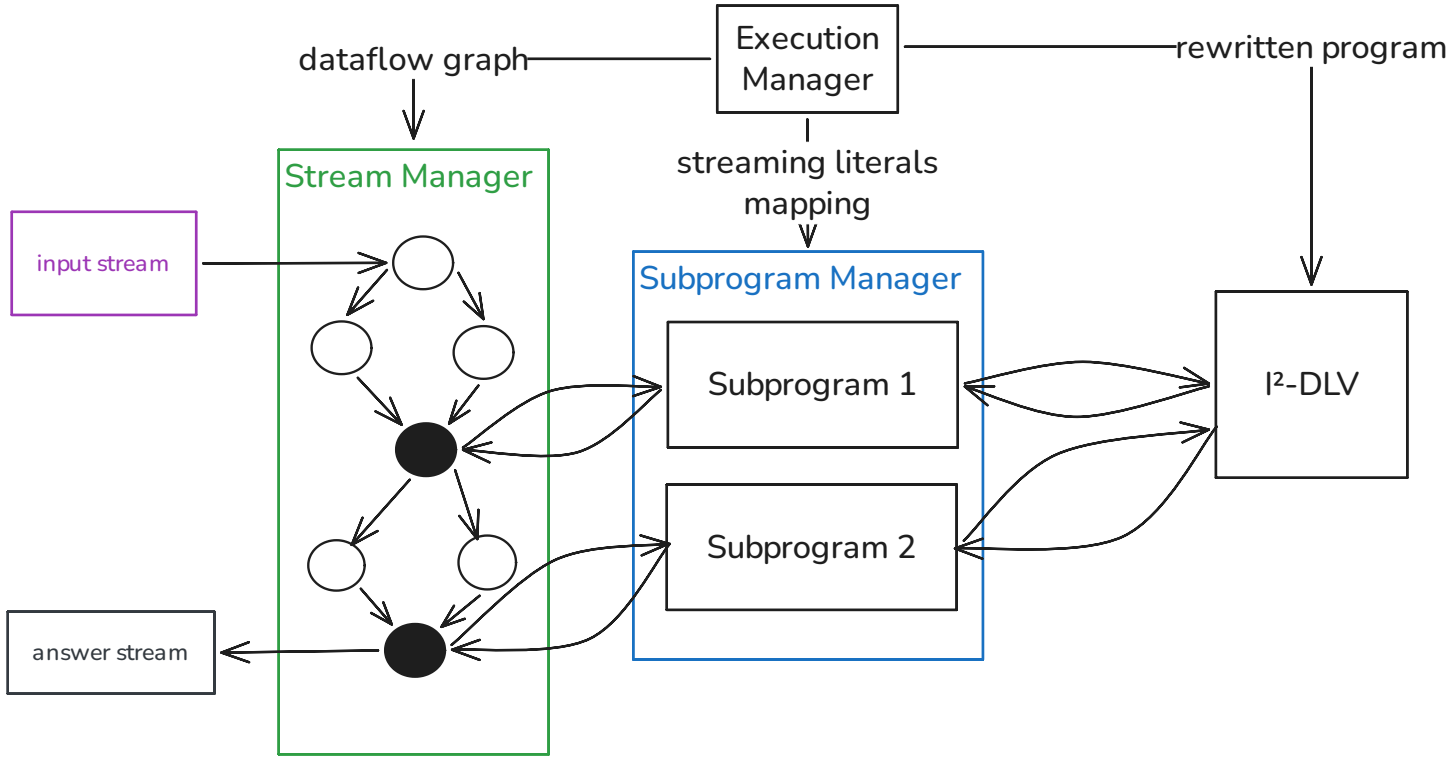
- P *stratified* by $\Pi_1, \ldots, \Pi_k$
  - $P_{(1)} = \{r \in P | r \text{ is not } \#temp\}$
- Stream $\Sigma = \langle S_0, \ldots, S_n \rangle$

- $\mathcal{P}(P, \Sigma)$: Persistent outcome over strata
- $\mathcal{P}(P, \Sigma) = \{a \in \mathcal{R}(P, \Sigma) | a \in S_n \vee \exists k, r : r \in P_{(1)} \wedge \langle \sigma, r \rangle \text{ for } \Sigma_{\Pi_k} \wedge \sigma(H(r)) = a\}$

- $\Sigma' = \langle S'_0, \ldots, S'_{n-1}, S_n \rangle$
- $S'_0 = \mathcal{P}(P, \langle S_0 \rangle)$
- $S'_i = \mathcal{P}(P, \langle S'_0, \ldots, S'_{i-1}, S_i \rangle)$
- Streaming model: $\mathcal{R}(P, \Sigma')$

Architecture

Top Level


Execution Manager
dataflow graph
rewritten program
Stream Manager
streaming literals mapping
input stream
Subprogram Manager
Subprogram 1
I²-DLV
Subprogram 2
answer stream

Execution Manager

- Program Rewriting
- Program Splitting
- Processing Ordering

Execution Manager - Program Rewriting

- Rewrites Non-degenerate streaming atoms
  - $p(t_1, \ldots, t_n)$ **op** in $\{d_1, \ldots, d_m\} \rightarrow p'(t_1, \ldots, t_n)$
    - **op** $\in$ { at least c, at most c, always, count c}
  - $p(t_1, \ldots, t_n)$ count X in $\{d_1, \ldots, d_m\} \rightarrow p'(t_1, \ldots, t_n, X)$

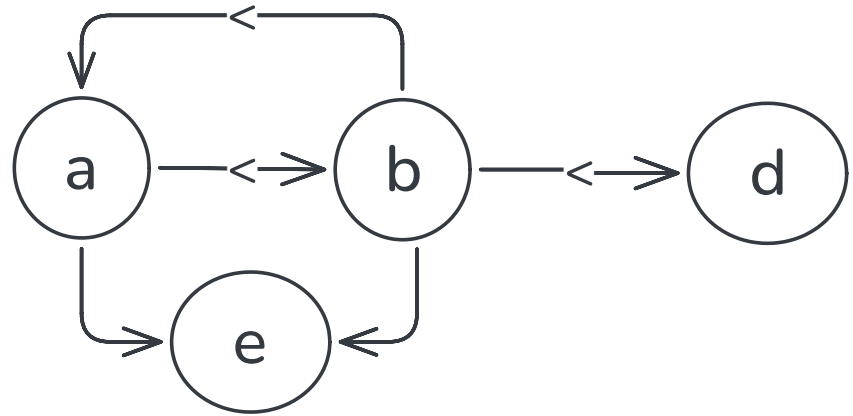
- $a(X) :- b(X)$ always in $[2]$.
- $b(Y) :- a(X)$ in $[1], Y = X + 1, c(Y)$.
- $d(X) :- b(X)$ at least 2 in $[4]$.
- $e(X, Y) :- a(X), b(Y)$.

- $a(X) :- b_{aux1}(X)$.
- $b(Y) :- a_{aux1}(X), Y = X + 1, c(Y)$.
- $d(X) :- b_{aux2}(X)$.
- $e(X, Y) :- a(X), b(Y)$.

- $b(X)$ always in $[2] \rightarrow b_{aux1}(X)$
- $a(X)$ in $[1] \rightarrow a_{aux1}(X)$
- $b(X)$ at least 2 in $[4] \rightarrow b_{aux2}(X)$

Execution Manager - Program Splitting

- Stream dependency graph G_P^{SD}
  - nodes: predicates p in rule heads $H(r)$
  - edges: p to q if $\exists r \in P : preds(H(r)) = \{q\} \wedge p \in preds(B(r))$
    - labelled with $<$ if p occurs in non-degenerate streaming literal

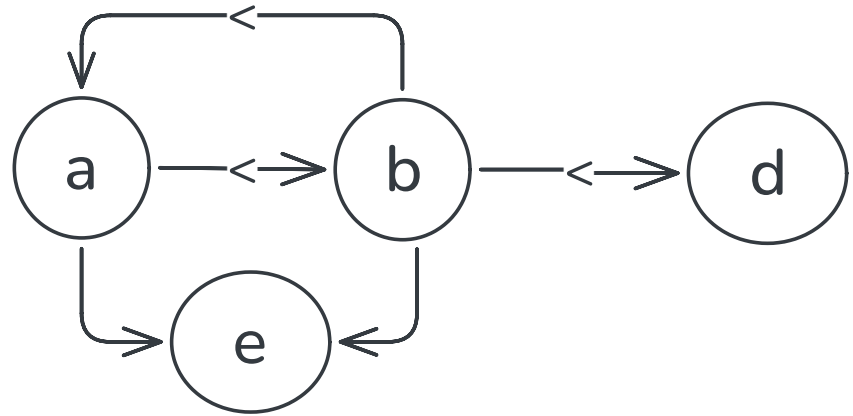


- $a(X) :- b(X)$ always in $[2]$.
- $b(Y) :- a(X)$ in $[1], Y = X + 1, c(Y)$.
- $d(X) :- b(X)$ at least 2 in $[4]$.
- $e(X, Y) :- a(X), b(Y)$.

Execution Manager - Program Splitting

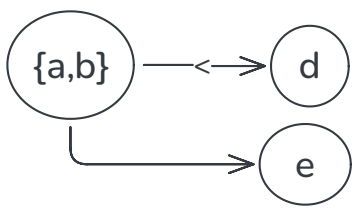
- Stream component graph G_P^{SC}
  - strongly connected components of G_P^{SD}

- $a(X) :- b(X)$ always in $[2]$.
- $b(Y) :- a(X)$ in $[1], Y = X + 1, c(Y)$.
- $d(X) :- b(X)$ at least 2 in $[4]$.
- $e(X, Y) :- a(X), b(Y)$.



Execution Manager - Process Ordering

- Stream component graph G_P^{SC}
- $A \prec B$: A *preceedes* B
  - there is a path from A to B with atleast 1 $<$ edge
- otherwise $A \approx B$: A *alongside* B
- ordering $C_1, \ldots, C_n$ for the nodes of G_P^{SC}
  - $\forall i, j : i < j \implies C_j \not\prec C_i$



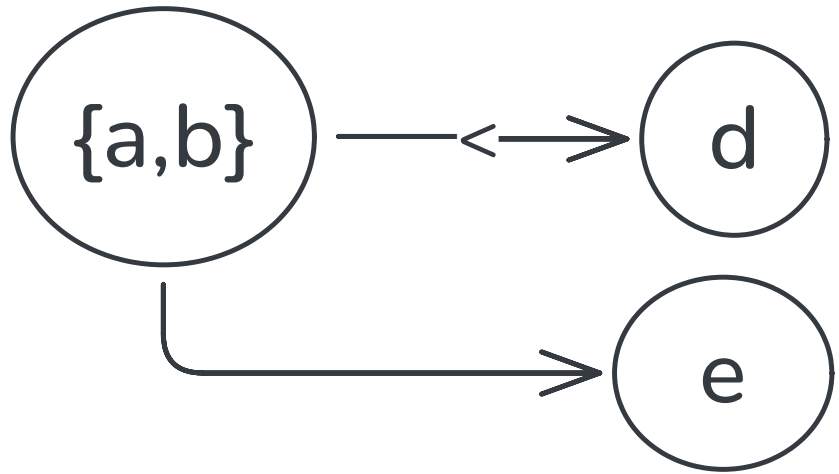
Possible orders

- $\{e\} \prec \{a, b\} \prec d$
- $\{a, b\} \prec \{e\} \prec d$
- $\{a, b\} \prec \{d\} \prec e$

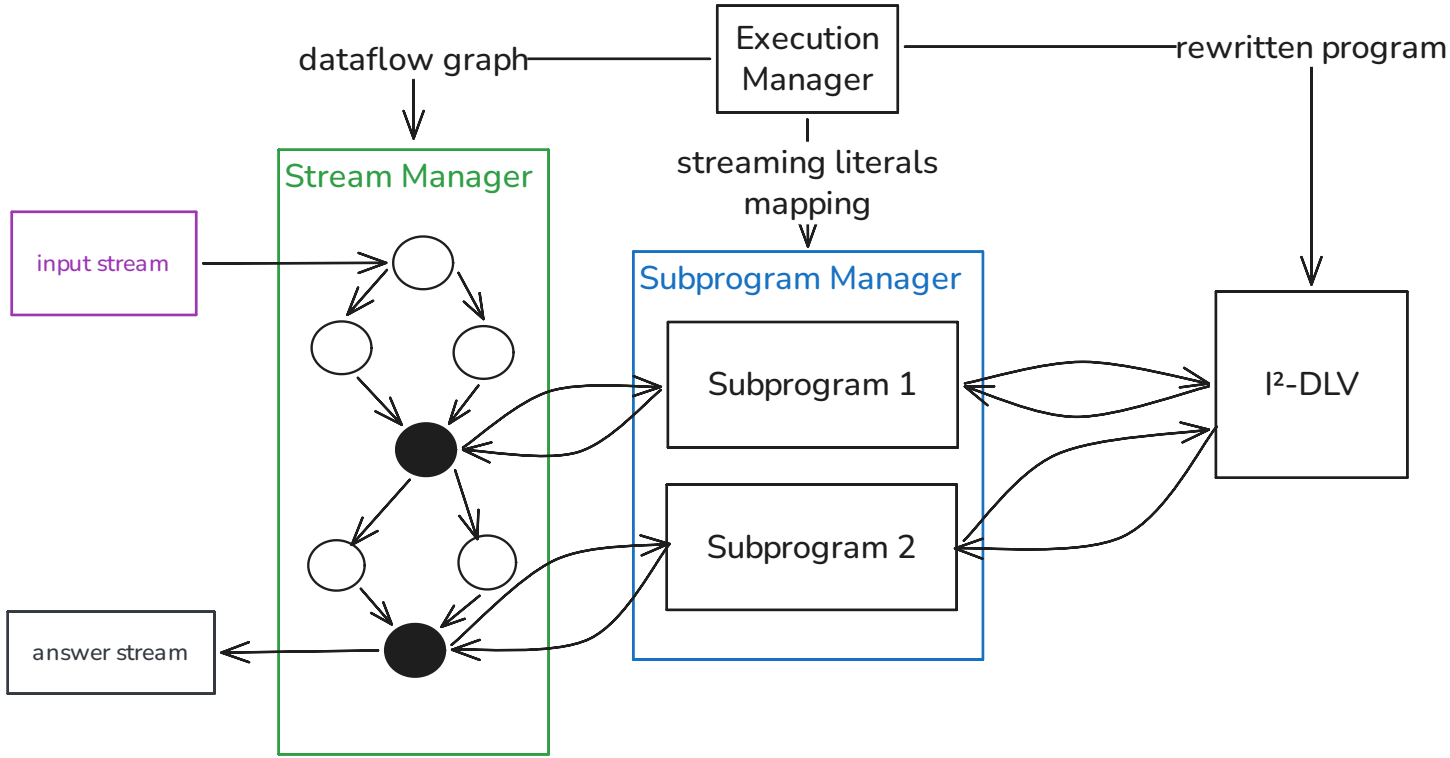
Execution Manager - Process Ordering

- ordering $C_1, \ldots, C_n$ for the nodes of G_P^{SC}
- collect pairwise nodes into macro nodes M
  - $i \leq j \leq k$: $M = \cup_j C_j$
  - *either* $i = k$
  - *or* $\forall j, j \neq k : C_j \approx C_{j+1}$
- for C_l, C_m with $l < m$
  - $C_l = \cup_{j_l} C_{j_l}, i_l \leq j_l \leq k_l$
  - $C_m = \cup_{j_m} C_{j_m}, i_m \leq j_m \leq k_m$
  - requires $k_l < i_m$

- $\{a, b\} \prec \{e\} \prec d$

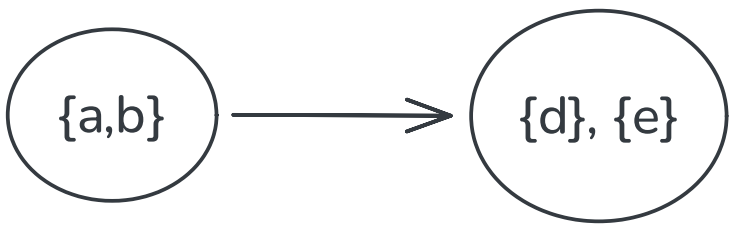


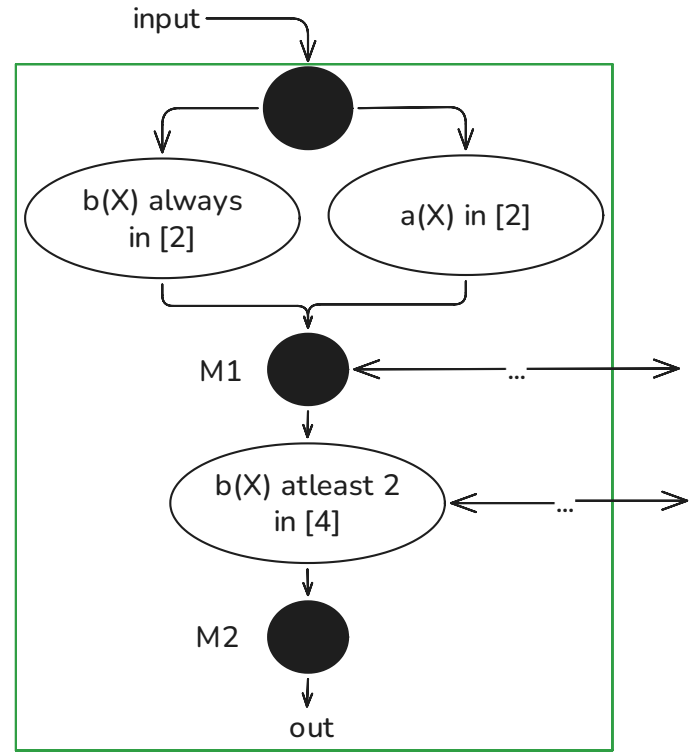
Stream Manager



Stream Manager

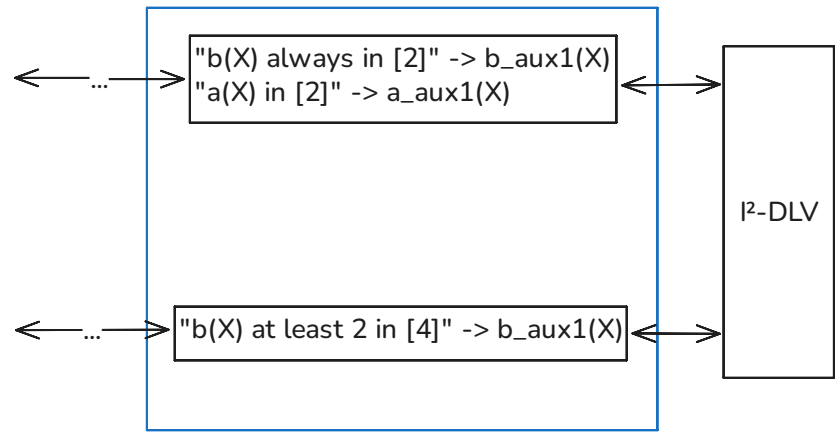
- $a(X) :- b(X)$ always in $[2]$.
- $b(Y) :- a(X)$ in $[1]$, $Y = X + 1, c(Y)$.
- $d(X) :- b(X)$ at least 2 in $[4]$.
- $e(X, Y) :- a(X), b(Y)$.



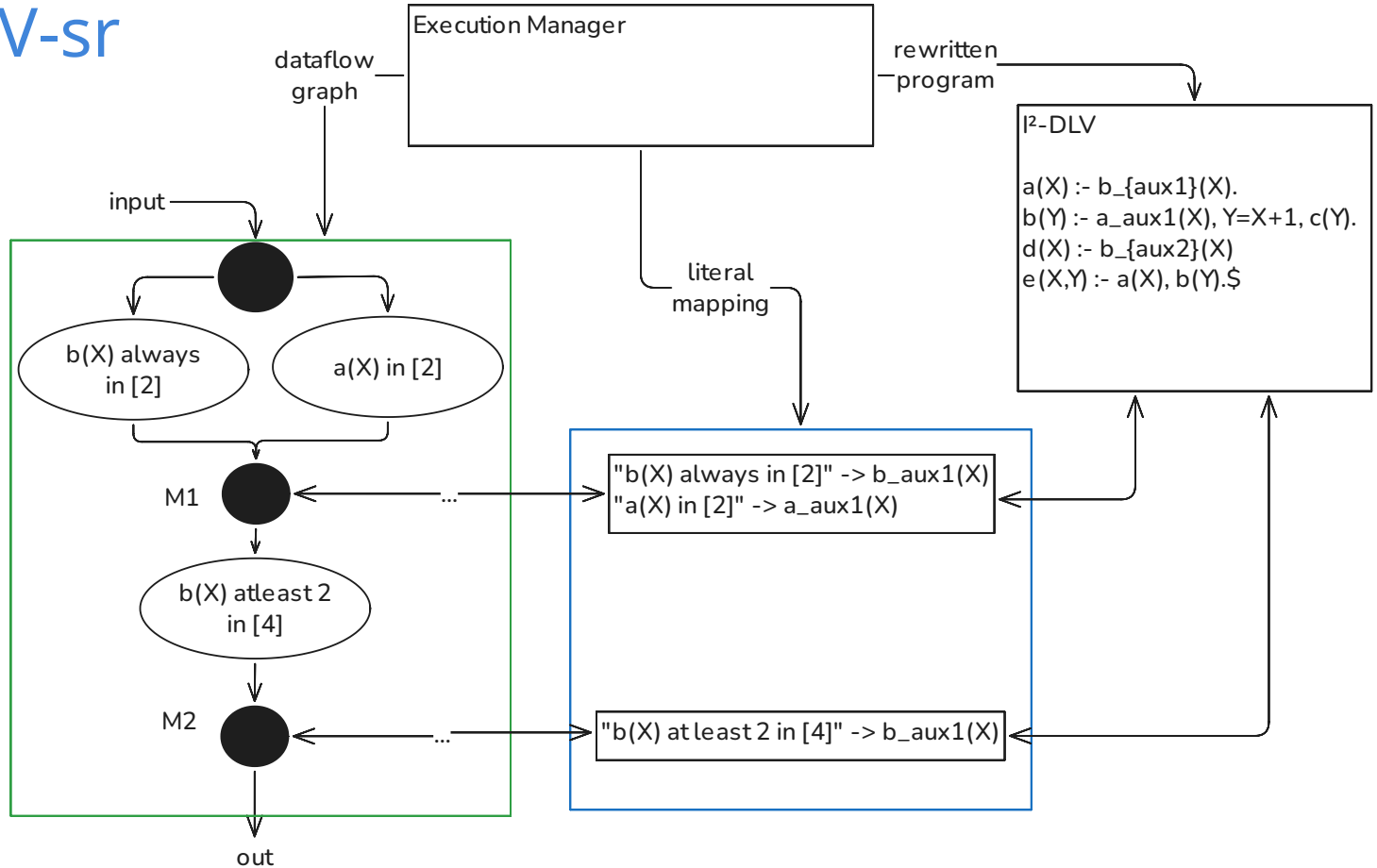


Subprogram Manager

- $a(X) :- b(X)$ always in $[2]$.
- $b(Y) :- a(X)$ in $[1]$, $Y = X + 1, c(Y)$.
- $d(X) :- b(X)$ at least 2 in $[4]$.
- $e(X, Y) :- a(X), b(Y)$.



I-DLV-sr


Execution Manager
dataflow graph
rewritten program
I²-DLV
a(X) :- b_{aux1}(X).
b(Y) :- a_aux1(X), Y=X+1, c(Y).
d(X) :- b_{aux2}(X)
e(X,Y) :- a(X), b(Y).$
input
literal mapping
b(X) always in [2]
a(X) in [2]
M1
...
"b(X) always in [2]" -> b_aux1(X)
"a(X) in [2]" -> a_aux1(X)
b(X) atleast 2 in [4]
M2
...
"b(X) at least 2 in [4]" -> b_aux1(X)
out

Thank you for your attention